Compilers Cheat Sheet, L1–5

Lexing (tokens, regex, NFA/DFA, lexer) and top-down LL(1) parsing.

1 The compiler

Phases: **lexing** (chars → tokens) → **parsing** (tokens → tree) → **semantic analysis** (types, scopes) → **code generation** → **optimisation**. Front end = analysis (source-specific), back end = synthesis (target-specific).

Compiler: translates once, then runs the target. Interpreter: executes the source directly. Assembler: assembly → machine code; linker joins objects.

Chomsky hierarchy

Type 3 regular = finite automaton = **lexer**. Type 2 context-free = pushdown automaton = **parser**. Type 1 context-sensitive (declare-before-use, type agreement) = handled by **semantic analysis**, not a bigger grammar. Type 0 = Turing machine: the limits (termination, optimal code are undecidable).

2 Tokens and regular expressions

Token ⟨name, attribute⟩ + position (line, column). **Lexeme**: the matched characters. **Pattern**: the rule. Whitespace, comments: matched, not returned.

Five core constructs (precedence: * > concat > |)

$a,\quad \varepsilon,\quad R \mid S,\quad RS,\quad R^*$

$L(R^*) = \{\varepsilon\} \cup L(R) \cup L(RR) \cup \cdots$

Sugar (no extra power): $R^+ = RR^*$, $R? = R \mid \varepsilon$, $[a\text{-}z]$, $[\hat{}x]$ (includes newline), . (excludes newline), $R\{n,m\}$, `"lit"`. Named abbreviations are never recursive.

Laws: $\varepsilon R = R$, $\emptyset R = \emptyset$, $R \mid \emptyset = R$, $(R^*)^* = R^*$, $R \mid R = R$.

Conflicts

- **Longest match** first (maximal munch): `classroom`, `ending`, `infinity` are identifiers.
- **Rule order** second, only on equal length: keywords before `ID`.
- **Catch-all** . last: error on one unknown char; never steals a token.

Classic patterns

`ID` $= [a\text{-}zA\text{-}Z_][a\text{-}zA\text{-}Z0\text{-}9_]^*$

`NUM` $= [0\text{-}9]^+(.[0\text{-}9]^+)?([eE][+-]?[0\text{-}9]^+)?$

`HEX` $= 0[xX][0\text{-}9a\text{-}fA\text{-}F]^+$

comment $=$ `/\*`$\big([\hat{}*] \mid *^+[\hat{}*/]\big)^**^+$`/` not `/\*.*\*/`

Sign: `-3` is two tokens; a sign inside NUM breaks `x-1`.

3 Finite automata

NFA: several moves per symbol, ε-moves; accepts if *some* path ends accepting. DFA: exactly one move per symbol; reject = all paths fail.

Method 1: Thompson (regex → NFA)

Build inside out: leaf a: 2 states, one edge. $R \mid S$: new start/accept, ε to both, ε out of both. R^*: new start/accept, ε start→accept (skip), ε back-edge (repeat). RS: A: ε-edge; B: merge accept of R with start of S. Properties: one start, one accept, no edge into start or out of accept, ≤ 2 out-edges.

states (B) $= 2(n_\ell + n_| + n_*) - n$. A: $2(n_\ell + n_| + n_*)$

edges (B) $= n_\ell + 4(n_| + n_*)$ A: $+n$.

$(0 \mid 1)^*1$: 10 states (A) or 9 (B). State your convention.

Methods 2–3: simulation, subset construction

$\mathrm{cl}(S) =$ states reachable from S by ε-edges

$\delta_D(S,c) = \mathrm{cl}\big(move(S,c)\big), \quad S_0 = \mathrm{cl}(\{q_0\})$

A DFA state is accepting if it contains an accepting NFA state; $\emptyset$ = dead state; stop when no new set appears. Write sets sorted.

NFA simulation $O(|S|\,|Q|)$ for Thompson NFAs

DFA scan $O(|S|), \quad \le 2^{|Q|}$ DFA states

L_k = "k-th symbol from the end is a": NFA $k{+}1$ states, every DFA 2^k.

Classic: $(a \mid b)^*abb$: Thompson 11 states (B) or 14 (A) → subset DFA 5 states $A..E$ → minimal 4 ($A \equiv C$).

Method 4: minimisation (table filling)

- Rule 1: mark (accepting, non-accepting) pairs. Witness ε.
- Rule 2, sweeps: mark (p,q) if some x sends it to a marked pair. Repeat until a sweep marks nothing.
- Merge unmarked pairs; **relabel all incoming edges**; for lexers, first separate different token labels.

Minimal DFA unique (Myhill–Nerode). Table $O(n^2)$ pairs; Hopcroft $O(n \log n)$.

Method 5: state elimination (DFA → regex)

$p \xrightarrow{A} q \xrightarrow{B} r$, loop L on $q \;\Rightarrow\; p \xrightarrow{AL^*B} r$ join parallel edges with |

Add a new final f; at the end start loop X, edge Y to f: X^*Y.

Method 6: Brzozowski derivatives

$\partial_a a = \varepsilon,\ \partial_a b = \emptyset,\ \partial_a \varepsilon = \partial_a \emptyset = \emptyset$

$\partial_a(R \mid S) = \partial_a R \mid \partial_a S, \quad \partial_a(R^*) = (\partial_a R)R^*$

$\partial_a(RS) = (\partial_a R)S \;\mid\; \nu(R)\,\partial_a S$ $\nu(R) = \varepsilon$ if R nullable, else $\emptyset$

$w \in L(R) \iff \partial_{a_n} \cdots \partial_{a_1} R$ nullable. Distinct derivatives = states of the minimal DFA.

Method 7: lower bounds, non-regularity

Pairwise distinguishable strings $w_1..w_k$ (each pair has a suffix z accepted after exactly one) ⇒ every DFA has $\ge k$ states; infinitely many ⇒ not regular ($(^n$, nested comments, indentation).

Pumping: $|w| \ge p \Rightarrow w = uvz,\ |uv| \le p,\ |v| \ge 1,\ uv^iz \in L\ \forall i$

To refute: take $w = a^pb^{\cdots}$ so $v = a^j$; pump $i = 0$ or 2, leave L.

4 From automata to a lexer

Merge all token NFAs under one new start (ε-edges); each accept state keeps its token label. Subset construction: a DFA state's label = the **highest-priority** (earliest rule) accepting NFA state it contains.

Longest match in a running lexer

- Run the DFA from the current position; at each accepting state record (position, token).
- Continue until stuck (no move or end of input).
- Emit the last recorded token; **push back** everything read after it; restart there.
- No accept ever recorded ⇒ lexical error (catch-all).

max pushback $= 1 +$ longest non-accepting run after an accept

Count every read, including the character that got the DFA stuck.

Quadratic trap: rules `ab`, `(ab)*c` on $(ab)^k$ read $\Theta(k^2)$ $(k(k{+}1))$ characters; Reps' memo of (state, position) failures makes it linear.

Beyond regular

Lexer states (start conditions) for comments, strings: `<COMMENT>`. Nested comments need a **counter**; Python indentation a **stack** of widths emitting INDENT/DEDENT; Fortran needs **lookahead** (`DO 10 I = 1,2` vs `1.2`); PL/I, C typedef need the parser.

Tools

flex/JFlex spec: definitions `%%` rules {action} `%%` user code. `yytext`, `yyleng`, `yylineno`; `yylex()` returns one token per call. Hand lexer bugs: `read()` without pushback (`sum+1`), committing too early (`x==y`), appending the quote before the test.

5 Grammars and parsing

$G = (N, T, P, S), \quad A \to \alpha, \quad \alpha \in (N \cup T)^*$

Leftmost derivation: always expand the leftmost non-terminal. Parse tree = derivation without order. **Ambiguous**: some string has two trees (two leftmost derivations).

Top-down = leftmost derivation, decides *which production*. Bottom-up = rightmost derivation in reverse, decides *when to reduce*.

Recursive descent: one procedure per non-terminal; `match(t)` consumes or errors; choose the alternative from one lookahead token. Left recursion = infinite recursion without consuming.

Unambiguous expressions (precedence by layers)

$E \to E{+}T \mid T,\quad T \to T{*}F \mid F,\quad F \to (E) \mid$ `id` left-assoc.

$E \to TE',\ E' \to +TE' \mid \varepsilon,\ T \to FT',\ T' \to *FT' \mid \varepsilon$ LL(1)

5 FIRST, FOLLOW

Nullable first: $A \to \varepsilon$, or $A \to Y_1..Y_k$ all nullable; iterate.

$\text{FIRST}(\alpha) = \{a : \alpha \Rightarrow^* a\beta\} \cup \{\varepsilon : \alpha \Rightarrow^* \varepsilon\}$

- Terminal: $\text{FIRST}(a) = \{a\}$. $X \to \varepsilon$: add ε.
- $X \to Y_1 Y_2 \cdots Y_k$: add $\text{FIRST}(Y_1) \setminus \varepsilon$; if Y_1 nullable add $\text{FIRST}(Y_2) \setminus \varepsilon$; … **stop at the first non-nullable**; all nullable $\Rightarrow$ add ε.
- **FOLLOW** 1: $\$ \in \text{FOLLOW}(S)$.
- 2: $A \to \alpha B \beta$: $\text{FIRST}(\beta) \setminus \{\varepsilon\} \subseteq \text{FOLLOW}(B)$.
- 3: $A \to \alpha B$, or $A \to \alpha B \beta$ with β nullable: $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$.

Every *occurrence* of B counts. Iterate to a fixed point; sets on an inclusion cycle are equal. ε is **never** in FOLLOW.

5 The LL(1) table and parser

- For $A \to \alpha$: put it in $M[A, a]$ for every $a \in \text{FIRST}(\alpha)$.
- If α is nullable: also in $M[A, b]$ for every $b \in \text{FOLLOW}(A)$ (incl. $).
- Empty cell = syntax error. Two productions in a cell = not LL(1).

LL(1) $\iff$ **for all** $A \to \alpha \mid \beta$

$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ no FIRST/FIRST

at most one of α, β nullable;

β nullable $\Rightarrow \text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ no FIRST/FOLLOW

Only non-terminals with ≥ 2 alternatives can clash; name the cell.

Stack algorithm (stack starts $ S, top X, lookahead a)

- $X = a = \$$: accept. $X = a$: pop, advance. X terminal $\neq a$: error.
- X non-terminal: $M[X, a] = X \to Y_1..Y_k$: pop X, push $Y_k, \ldots, Y_1$ (Y_1 on top), output the production; empty cell: error.

Trace columns: stack | input | action. The outputs are the leftmost derivation.

5 When a grammar is not LL(1)

$A \to A\alpha \mid \beta \;\Rightarrow\; A \to \beta A',\ A' \to \alpha A' \mid \varepsilon$ left recursion

General: $A \to A\alpha_1|..|A\alpha_m|\beta_1|..|\beta_n \Rightarrow A \to \beta_j A',\ A' \to \alpha_i A' \mid \varepsilon$. Indirect ($S \to Aa$, $A \to Sd$): order $A_1..A_n$; for each i, inline every A_j ($j < i$) that starts a production of A_i, then remove direct recursion.

$A \to \alpha\beta_1 \mid \alpha\beta_2 \;\Rightarrow\; A \to \alpha A',\ A' \to \beta_1 \mid \beta_2$ left factoring

ε-removal: for each production add all versions with any subset of nullable symbols deleted; drop $A \to \varepsilon$; if S was nullable add $S' \to S \mid \varepsilon$.

Ambiguity cannot be removed by these: dangling else $S \to iEtSS' \mid a$, $S' \to eS \mid \varepsilon$ has $M[S', e] = \{S' \to eS,\ S' \to \varepsilon\}$; resolve by choosing $S' \to eS$ (else to the nearest `if`).

Always **re-check** LL(1) after a transformation. They keep the language, change the trees (associativity).

Worked: the expression grammar

$E \to TE',\ E' \to +TE' \mid \varepsilon,\ T \to FT',\ T' \to *FT' \mid \varepsilon,\ F \to (E) \mid \texttt{id}$

Nullable: E', T'.

	FIRST	FOLLOW
E	{(, id}	{), $}
E'	{+, ε}	{), $}
T	{(, id}	{+,), $}
T'	{*, ε}	{+,), $}
F	{(, id}	{*, +,), $}

	id	+	*	(	)	$
E	TE'			TE'		
E'		$+TE'$			ε	ε
T	FT'			FT'		
T'		ε	$*FT'$		ε	ε
F	id			(E)		

Why $E' \to \varepsilon$ under) and $: E' is nullable, $\text{FOLLOW}(E') = \{), \$\}$. Why $\text{FOLLOW}(T) \ni +$: $E' \to +TE'$ puts $\text{FIRST}(E') \setminus \varepsilon$ after T, and E' nullable adds $\text{FOLLOW}(E')$.

Trace of `id+id`

stack (top right)	input	action
$ E	id+id$	$E \to TE'$
$ $E'T$	id+id$	$T \to FT'$
$ $E'T'F$	id+id$	$F \to$ id
$ $E'T'$id	id+id$	match
$ $E'T'$	+id$	$T' \to \varepsilon$
$ E'	+id$	$E' \to +TE'$
$ $E'T+$	+id$	match
$ $E'T$	id$	$T \to FT'$
$ $E'T'F$	id$	$F \to$ id
$ $E'T'$id	id$	match
$ $E'T'$	$	$T' \to \varepsilon$
$ E'	$	$E' \to \varepsilon$
$	$	accept

5 Recursive descent: grammar ↔ code

Grammar → code

- $\text{SEL}(A \to \alpha) = \text{FIRST}(\alpha) \setminus \varepsilon$, plus $\text{FOLLOW}(A)$ if α is nullable: the tokens that select this production.
- One procedure per non-terminal, one global lookahead `la`. `match(t)`: `la==t`? advance : error.
- `void A(){ if (la in SEL1) {body1} else if (la in SEL2) {body2} else error(); }`
- Body, left to right: terminal $t \to$ `match(t);` non-terminal $B \to$ `B();` $\varepsilon \to$ nothing, just return.
- $A' \to t\,\beta\,A' \mid \varepsilon$ may be coded `while (la==t) {match(t); body of` β`}`.
- Driver: `la=next(); S(); if (la!=EOF) error();`

`void Ep(){ if (la=='+') {match('+'); T(); Ep();} else if (la==')'||la==EOF) {} else error(); }` for $E' \to +TE' \mid \varepsilon$, $\text{FOLLOW}(E') = \{), \$\}$.

Code → grammar

- Procedure = non-terminal; start symbol = the one the driver calls.
- Each `if`/`else if` branch = one production; its `match`/call sequence, in order, = the right side; the condition = its SEL.
- Empty branch `{}` = $A \to \varepsilon$; its condition tokens are $\text{FOLLOW}(A)$, not symbols of the right side.
- A condition token is in the right side **only if** a `match` of it follows.
- `while (la==t) {...}` inside `A()` = a new $A' \to t \ldots A' \mid \varepsilon$ at that point of the right side.
- Check: conditions pairwise disjoint in every procedure $\iff$ LL(1). Trace: indent per call, note the deciding token; only `match` consumes.

Bugs to spot

`A(){A();..}`: left recursion, nothing consumed, stack overflow. A `match` of a FOLLOW token inside the callee steals it from the caller: `(id)` rejected. `else {}` with no FOLLOW test defers the check: without `la==EOF` in the driver, `id id` is accepted.

Quick answers

Left-recursive or ambiguous $\Rightarrow$ never LL(1). $\varepsilon \in$ FOLLOW: never. $\text{FOLLOW}(S) \supseteq \{\$\}$ if S occurs on a right side ($S \to (S) \mid \varepsilon$: {), $}). Same FIRST $\not\Rightarrow$ same FOLLOW. Disjoint FIRSTs $\not\Rightarrow$ LL(1) (FIRST/FOLLOW). FIRST/FOLLOW are computable for any CFG. Rule order never beats length. One failed NFA path does not reject.

Traps

Thompson counts depend on the convention. Relabel incoming edges after merging. `.*` in a comment overruns to the last `*/`. Stopping the FIRST scan too early or too late. Only literal $A \to \varepsilon$ under FOLLOW (wrong: every nullable right side). One-sweep FOLLOW on a cycle. Pushing a right side in the wrong order.